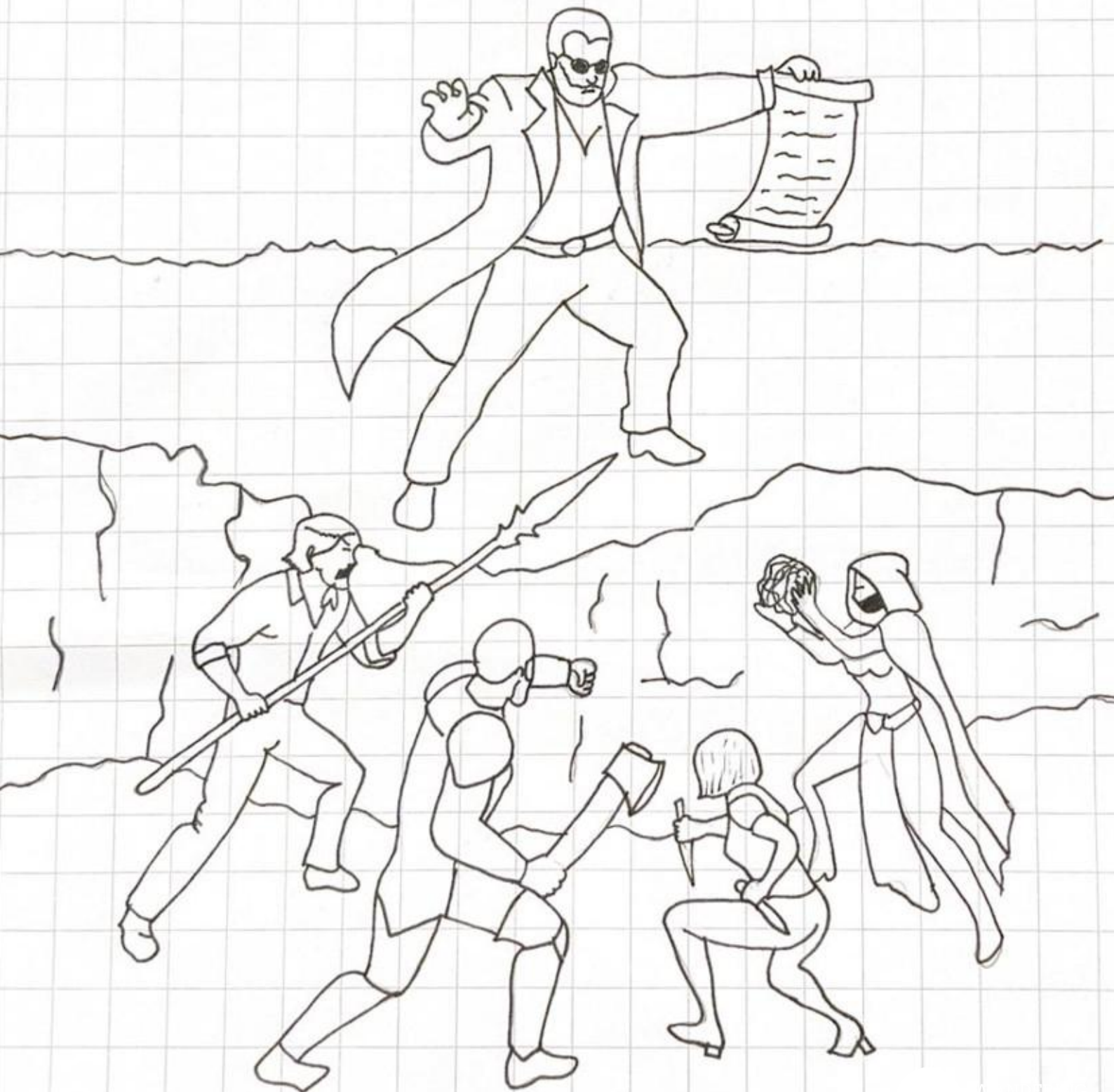


Punchline:

A Literal Adventure

Technical Design Document 2025/2026



1. Sommario

2. Revision History	4
3. Project Goal.....	5
4. Provided Services (Beside the Game)	6
4.1 Website.....	6
4.2 Community Channels.....	6
4.3 Stores and platform services	6
4.4 Maintenance and updates.....	6
4.5 Additional Services.....	6
5. Client side	7
5.1 Hardware Requirements	7
5.2 Software Requirements.....	7
6. Workload Estimation	8
6.1 Vertical Slice / Closed Beta.....	8
6.2 Full release	8
7. General Architecture	9
7.1 Frontend.....	9
7.1.1 Game client	9
7.1.2 Distribution Platforms.....	10
7.1.3 Discord (Full release).....	10
7.1.4 Instagram and Tiktok (Full release).....	10
7.1.5 Website (Full release).....	11
7.2 Backend.....	11
7.2.1 Source Control - Git	11
7.2.2 CI/CD Pipeline (Beta and forward)	12
7.2.3 Unity Game Services (UGS).....	12
7.2.4 Hosting and Infrastructure Summary.....	13
8. External Services	14
8.1 Payments	14
8.2 Authentication.....	14
8.3 Social media.....	14

8.4	Legal Support	15
8.5	Achievements	15
9.	Connection	16
9.1	Global Infrastructure	16
9.1.1	Development Team Connectivity	16
9.1.2	Player Session Infrastructure	17
9.2	Network Requirements	17
10.	Delivery	19
10.1	Estimated Delivery Time	19
10.2	Delivery Platform.....	20
10.3	Post-lunch plans.....	20
11.	Cost Estimation.....	21
11.1.1	Staff intern.....	21
11.1.2	External services/outourcing	21
11.1.3	Workspace.....	22
11.1.4	One time setup costs.....	22
11.2	Final cost evaluation	22

2. Revision History

When	Who	What
10/04/2026	Federico Allegretti	Project Goal (Initial Draft)
11/04/2026	Federico Allegretti	Provided Services (Initial Draft)
12/04/2026	Federico Allegretti	Client Side (Initial Draft)
13/04/2026	Federico Allegretti	Workload Estimation (Initial Draft)
14/04/2026	Federico Allegretti	General Architecture (Initial Draft)
14/04/2026	Leandro Jolldashi	Cost evaluation (Initial Draft)
14/04/2026	Leandro Jolldashi	Project Goal semi-final text
16/04/2026	Leandro Jolldashi	Provided Services complete
16/04/2026	Federico Allegretti	Client side, Workload estimation, General Architecture, Connection (Finished)
17/04/2026	Nicola Cozzolino	External Services
16/04/2026	Leandro Jolldashi	Gant Diagram
16/04/2026	Leandro Jolldashi	Completed Delivery
17/04/2026	Federico Allegretti	Connection Graph
17/04/2026	Leandro Jolldashi	Completed Cost Evaluation

3. Project Goal

Punchline is an online cooperative beat 'em up game made to give players a smooth and fun multiplayer experience.

The gameplay is based on coordination, timing, and precision during combat. From a technical point of view, the main goal of the project is to let players play together smoothly through a peer-to-peer connection. We want to reduce latency and jitter as much as possible, so the game can stay fast and responsive. Since Punchline is based on quick actions and cooperation, network quality is very important for the overall experience and a good connection helps make combat clearer, fairer, and more enjoyable for players.

For this reason, the technical solution must focus on stable online sessions, low system overhead, and an easy way for players to join cooperative matches.

The goal of the beta is to test the main online experience, the stability of the distribution process, and the reliability of the services needed to make the game available to players. Other features such as extra content, trophies, or DLCs may be added later, but they are not the focus of this technical plan.

The project will use platform services such as Steam, PlayStation Network, and Xbox Live when needed. These services will help with distribution, authentication, updates, and player access to the game. In this way, the team can spend more time improving the gameplay experience instead of building every support feature from scratch.

At the same time, Punchline also needs a technical workflow that supports continuous integration and delivery, so new builds can be released more easily during development and testing.

Finally, the project infrastructure must be easy to maintain and must support safer releases through automated tests during the build process. This helps reduce critical problems with new versions of the game.

Overall, the goal is to build a stable, accessible, and scalable technical base for the open beta of Punchline, while keeping the possibility to improve and expand the game in the future.

4. Provided Services (Beside the Game)

4.1 Website

Punchline will have an official website where players can find general information about the game, links to the main online stores, and links to the project social channels.

The website will also include a simple FAQ or forum section where players can read common information, discuss the game, and report technical or legal issues to the team. This website will work as a central point for communication between the project and its community.

4.2 Community Channels

Punchline will also use community channels such as YouTube, Instagram and TikTok to share updates, trailers, and news about the game.

These channels will help the team promote the project and collect feedback from players more quickly. They will also help keep the community active during the beta phase and after future updates.

4.3 Stores and platform services

The game will be distributed through major digital platforms such as Steam, Nintendo eShop, PlayStation Store, and Xbox Store. These platforms will help manage game distribution, updates, visibility, and access to platform-related services. They will also give the game more visibility thanks to their internal store pages and community features.

4.4 Maintenance and updates

During the beta and in later versions, Punchline will receive updates to fix bugs, improve stability, and improve the overall player experience.

Further updates may also add new content or extra features when needed. This will help keep the game more reliable and interesting for players over time.

4.5 Additional Services

When needed, Punchline will rely on platform services to simplify player access, authentication, and other support functions connected to the game. Using these services will reduce the amount of non-core systems that the team has to build directly. This lets the team focus more on the gameplay experience and on the quality of the game itself.

5. Client side

5.1 Hardware Requirements

The hardware requirements listed below reflect the minimum and recommended configurations for running Punchline on PC. Full details and rationale are covered in the **Game Design Document, section 3.3 System requirements**.

Component	Minimum	Recommended
CPU	Intel Core i5-6600K / AMD Ryzen 5 1600	Intel Core i7-12700K / AMD Ryzen 7 5800X
GPU	NVIDIA GTX 1060 6GB / AMD RX 580	NVIDIA RTX 3070 / AMD RX 6800
RAM	8 GB	16 GB
Network	Broadband connection	Broadband connection
Audio	Required	Required
OS	Windows 11	Windows 11

The player will also need a controller (PS5 Dualsense, Xbox Controller, Steam Controller) or a mouse and keyboard combo. The player will mostly use the keyboard, but the mouse is necessary for the UI operations.

5.2 Software Requirements

The following software must be present on the player's machine for the game to run correctly.

Runtime dependencies: Unity-built applications on PC bundle most of their runtime dependencies within the executable package. No separate Unity installation is required by the player. However, the following system-level components must be present:

- **Microsoft Visual C++ Redistributable (2015-2022):** required by Unity's native plugin layer. Distributed automatically with the game installer if not already present on the system.
- **DirectX 11 or higher:** required by Unity's rendering pipeline on Windows. Present by default on all Windows 11 installations.
- **.NET runtime:** bundled within the Unity build, no separate installation required.

Platform software: For the vertical slice, no platform client (Steam, Epic Games Store, etc.) is required, as the game is distributed as a standalone PC build. At full release, players will need to install the chosen platform client to access the game.

Operating system configuration: The players need to have a machine with an installation of Windows 11 to play the game.

6. Workload Estimation

This section estimates the expected infrastructure and service load across two distinct phases: the vertical slice and the full release. The two phases differ significantly in player volume, active services, and operational complexity.

6.1 Vertical Slice / Closed Beta

The vertical slice expected target concurrent players is low and controlled. At this scale, all active services operate well within their free tier limits, and no paid infrastructure is required.

6.2 Full release

At full release, the player base and active services expand significantly. The web presence becomes active, platform stores go live, and UGS usage scales with the player count. Cost estimates below are based on a conservative projection of 500–2,000 concurrent players at peak, which is a realistic target for an indie cooperative title in its first months after launch.

Service	Expected Load	Pricing Model	Estimated Monthly Cost
Unity Relay	500–2,000 concurrent players	UGS pay-as-you-go (per GB relayed)	~€10–40
Unity Lobby	200–800 active lobbies	UGS pay-as-you-go (per lobby-hour)	~€5–15
Static website (CDN)	50,000–200,000 requests / month	Free tier on Cloudflare/Netlify	Free
Community forum (VM)	Low-to-moderate traffic	Cloud VM (e.g. DigitalOcean Droplet)	€0–25
Forum database	Persistent storage	Managed PostgreSQL (e.g. Supabase free/pro)	€0–25
CI/CD pipeline	~500–1,000 build minutes / month	GitHub Actions free tier or pro	€0–4

UGS relay costs scale with the volume of data relayed, so sessions where direct peer connections are successfully negotiated via hole-punching generate no relay cost, as traffic bypasses the relay entirely.

7. General Architecture

Punchline's infrastructure is designed to be simple and reliable. The game is built in Unity and relies on peer-to-peer networking for its multiplayer sessions, which means the architecture does not require a dedicated game server to manage real-time state. Instead, the infrastructure focuses on the game client pipeline, a robust development workflow, and the supporting web services that will accompany the full release.

For the current phase, a vertical slice targeting PC, the active infrastructure is intentionally minimal: a Unity build distributed directly to testers, a CI/CD pipeline for automated builds and testing, and Unity's managed relay service for session connectivity. The web presence (public website and community forum) is part of the planned full-release infrastructure and is described here for completeness, but it is not in scope for the vertical slice.

The overall system is divided into two main areas: the front end, which covers all surfaces the player interacts with directly, and the back end, which covers the development pipeline, hosting services, and all supporting systems that keep the game and its web presence operational.

7.1 Frontend

The front end contains every player-facing touchpoint: the game client itself, the public website, and the community forum. For the vertical slice, only the game client is active.

7.1.1 Game client

The game is developed in Unity and targets PC (Windows) as its primary platform for the vertical slice. The client is responsible for all gameplay logic, rendering, input handling, audio, and network communication. The Unity project is structured into clearly separated modules (core gameplay systems, networking layer, UI, and audio) to support parallel development and simplify the integration of automated tests during the build process.

Multiplayer connectivity is handled through Unity Netcode for GameObjects (NGO), Unity's first-party networking framework. NGO manages object state synchronization, remote procedure calls (RPCs), and the host/client session model across all connected peers.

For the vertical slice, the connection model uses a host-client topology: one player acts as the session host while others connect as clients. This is the standard configuration for cooperative P2P sessions in NGO and does not require a dedicated server.

Player discovery and session establishment are handled through **Unity Lobby** and **Unity Relay**, both part of Unity Gaming Services (UGS).

- **Unity Lobby** allows players to create and join named sessions without exposing direct IP addresses.
- **Unity Relay** provides a managed relay server that forwards traffic between peers when a direct connection cannot be established due to NAT restrictions. More importantly, this relay only handles connection routing and does not process game state, preserving the P2P nature of the architecture.

For the full release, the game client will be distributed across all major digital storefronts (Steam, Epic Games Store, GOG Galaxy, PlayStation Store, Nintendo eShop, Xbox / Microsoft Store), leveraging each platform's native SDK for authentication, updates, and player session management. This is not in scope for the vertical slice.

7.1.2 Distribution Platforms

At full release, Punchline will be distributed across all major digital storefronts. Each platform acts as a front-end surface for the player: it handles the game's public store page, purchase flow, download, and update delivery. The team is responsible for maintaining the store presence on each platform, including descriptions, screenshots, trailers, and pricing, through the respective developer portals.

- **Steam** (PC - Windows): store page managed via the Steamworks Partner portal. Builds are delivered through SteamPipe and updates are distributed natively through the Steam client.
- **Epic Games Store** (PC - Windows): store page managed via the Epic Developer Portal. Builds are delivered through the BuildPatch Tool (BPT).
- **GOG Galaxy** (PC - Windows, DRM-free): store page managed via the GOG Developer Portal. Unlike Steam and Epic, GOG does not enforce DRM, so the distributed build must be self-contained. Updates are delivered through the GOG Galaxy client or as standalone installers.
- **PlayStation Store** (PS4 / PS5): store page managed via PlayStation Partners. Builds require mandatory certification by Sony before publication. The store page is region-aware and may require localized assets.
- **Nintendo eShop** (Nintendo Switch): store page managed via the Nintendo Developer Portal. Builds require certification by Nintendo. The eShop presentation is controlled by Nintendo's storefront layout with limited customization.
- **Xbox / Microsoft Store** (Xbox Series X|S, Xbox One, PC): store page managed via Partner Center. Builds require certification by Microsoft. The Microsoft Store also covers the PC distribution channel within the Xbox ecosystem.

For the vertical slice, none of these storefronts are active. Distribution is handled directly via a packaged PC build shared with testers.

7.1.3 Discord (Full release)

A dedicated Discord server will serve as the primary real-time community hub for Punchline. It will complement the forum by providing a faster, more informal space for player communication, bug reporting during beta sessions, and direct interaction with the development team. Discord requires no hosting or backend infrastructure from the team. The server will be organized into channels covering announcements, general discussion, feedback, and support. A permanent invite link will be prominently featured on the static website.

7.1.4 Instagram and Tiktok (Full release)

Official Instagram and TikTok profiles will be maintained as the primary social media channels for Punchline's visual communication. They will be used to publish development updates, gameplay clips, artwork, and release announcements. Instagram and TikTok require no infrastructure management from the development team. Content will be produced and scheduled as part of the broader marketing effort surrounding the full release. The profiles link will be featured on the static website alongside other community channels.

7.1.5 Website (Full release)

7.1.5.1 façade

The public-facing website will serve as the primary marketing and information surface for Punchline. It will be a fully static site containing game description, trailers, screenshots, press kit, FAQ, and platform download links, pre-built and served through a Content Delivery Network (CDN) for fast global load times and minimal maintenance overhead.

The site will be authored using a static site generator (such as Hugo or Astro) and deployed automatically on every merge to the main branch through the CI/CD pipeline. No database or application server is required for this layer.

7.1.5.2 Forum

The forum will be the dynamic component of the web presence, providing players with a space to report bugs, share feedback, and engage with the community. It will be built on a self-hosted or managed open-source solution (such as Discourse or Flarum), deployed on a cloud virtual machine or managed container service. The forum is the only web component that requires active database management, backup policies, and moderation tooling.

All the manpower for the development and maintenance of the dynamic website could be outsourced from a third party, or completely.

7.2 Backend

The back end supports development operations, automated delivery, and, at full release, the web services described above. For the vertical slice, the active back-end components are source control, the CI/CD pipeline, and Unity Gaming Services.

7.2.1 Source Control – Git

All source code including the Unity project, any tooling, and configuration scripts are maintained in a Git repository hosted on a managed platform (GitHub or GitLab). The repository follows a branch-based workflow:

- **main:** Stable, production-ready code; protected branch requiring pull request review before merge.
- **develop:** Active integration branch for ongoing work.
- **Feature branches:** Short-lived branches created per task or feature, merged into **develop** via pull requests.
- **Release branches:** Created from **develop** when preparing a distributable build for testing or submission.

7.2.2 CI/CD Pipeline (Beta and forward)

A continuous integration and continuous delivery pipeline are configured to run automatically on every push to the repository. The pipeline is managed through a dedicated CI/CD service, such as GitHub Actions or GitLab CI, and executes the following stages on every build:

1. **Build:** The Unity project is compiled into a PC executable using Unity's command-line build interface. The static website is also built at this stage when applicable.
2. **Automated Testing:** A suite of automated tests is executed as part of every build using the Unity Test Framework. This includes unit tests for core game systems (input handling, combat state machines, synchronization logic) and integration tests for networking layer interactions. Builds that fail any test are automatically rejected and do not proceed to the next stage.
3. **Artifact Packaging:** Passing builds are packaged into distributable archives ready for internal distribution to testers.
4. **Deployment:** On successful builds targeting specific branches, deployment is triggered automatically. For the vertical slice, this means uploading the packaged build to a shared distribution point accessible to testers. At full release, this stage will also cover deployment to platform developer portals and CDN updates for the website.

7.2.3 Unity Game Services (UGS)

Unity Gaming Services provides the cloud infrastructure needed to support multiplayer sessions without building proprietary backend systems. The services used are:

Service	Purpose
Unity Lobby	Session creation, discovery, and player matchmaking
Unity Relay	Add peer-to-peer multiplayer without dedicated servers.

Both services are managed by Unity and require no server-side code from the development team. They are accessed directly from the game client via the UGS SDK and are available on Unity's free tier for the scale of the vertical slice and early beta.

7.2.4 Hosting and Infrastructure Summary

Service	Technology	Scope
Game Client	Unity	Vertical Slice + Full release
Multiplayer	Unity NGO + Unity Relay	Vertical Slice + Full release
Player Lobbies	Unity Lobby (UGS)	Vertical Slice + Full release
CI/CD Pipeline	Github Actions / Gitlab CI	Vertical Slice + Full release
Source Control	Github / Gitlab	Vertical Slice + Full release
Static Website	CDN (Cloudflare, Netlify, or Vercel)	Full release only
Community Forum	CloudVM + Discourse/Flarum	Full release only
Forum database	Managed PostgreSQL	Full release only
Discord Server	Discord (third-party managed)	Full release only
Instagram Profile	Instagram (third-party managed)	Full release only

8. External Services

This chapter describes the external services required to support the development, release, and post-release lifecycle of *Punchline*. The associated costs of these services are detailed in **Section 11, Cost Estimation**. These services are essential not only to ensure a successful launch, but also to support earlier phases such as the **closed beta** and **open beta**, as well as future updates and promotional activities.

8.1 Payments

At release, *Punchline* will be available on Steam and Nintendo Switch, with plans for additional platforms, as described in **Section 3.2.1 Target Platforms** of the **GDD**.

Payment processing will be managed directly through the digital distribution platforms used for release. These platforms will handle transactions securely and deduct their standard fees from sales revenue.

8.2 Authentication

Player authentication will rely primarily on the authentication systems provided by the distribution platforms themselves. In the initial release, this will include:

- **Steam authentication** for the PC version.
- **Nintendo Switch Online** authentication for the console version.

For future releases on additional platforms, based on the priority explained in the section **3.2.1 Target Platforms** of the **GDD**, authentication will be adapted to the relevant ecosystem, such as:

- **Microsoft account** for Xbox releases.
- **PlayStation Network** (PSN) for PlayStation releases.
- **Google account** for a possible mobile release.

8.3 Social media

Social media will play a central role in increasing the visibility of **Punchline**, especially considering the indie nature of the project. Through the creation of engaging and shareable content, the social media presence will help build an audience both for the final product and for the studio itself, **Wandering Strangers**.

The studio's social channels will not be dedicated exclusively to *Punchline* but will also serve as a broader communication platform for future titles, expansion content, development updates, and community engagement.

They will be used to:

- Attract attention to the game through short-form promotional content.
- Recruit play testers for the closed and open beta phases.
- Share development updates, trailers, and announcements.
- Maintain long-term visibility for future projects.

The main platforms will include:

- **TikTok and Instagram:** These channels will focus on short videos showcasing the game's core mechanics, visual identity, and distinctive features. Content will follow relevant indie development trends to maximize reach and engagement.

- **Twitch and YouTube:** By providing early access, review copies, or beta access to streamers and content creators focused on gaming, the studio aims to generate interest in the game, broaden its audience, and collect additional feedback from players and viewers.
- **Wandering Strangers YouTube Channel:** This channel will host longer-form content such as gameplay trailers, cinematic trailers, developer updates and logs, and promotional videos for future expansions and projects.

Initially, all social media platforms will be managed directly by the development team. Due to limited workforce and resource constraints, only **Instagram and TikTok** pages will be actively maintained during the early stages of the project.

Once a sufficient level of audience engagement and interest has been achieved, the team will transition to a more structured communication strategy by onboarding dedicated professionals (e.g., social media managers and content creators).

This will enable the expansion of existing channels, the activation and consistent management of the YouTube channel, and the production of higher-quality and more frequent content.

8.4 Legal Support

Wandering Strangers will rely on legal support to ensure that the project is properly structured and compliant with applicable regulations. Legal assistance will be required for the following areas:

- **Terms of Service (ToS):** to draft clear and legally sound terms governing the use of the game and related services.
- **GDPR compliance:** to define a proper data protection framework for the management of analytics, user data, and any other personal information collected during gameplay, testing, or community activities.
- **Contracts:** to regulate the relationship with freelance artists, contractors, and social media managers.
- **Trademark protection:** to safeguard the intellectual property of **Punchline, Wandering Strangers**, and any future projects connected to the studio.

8.5 Achievements

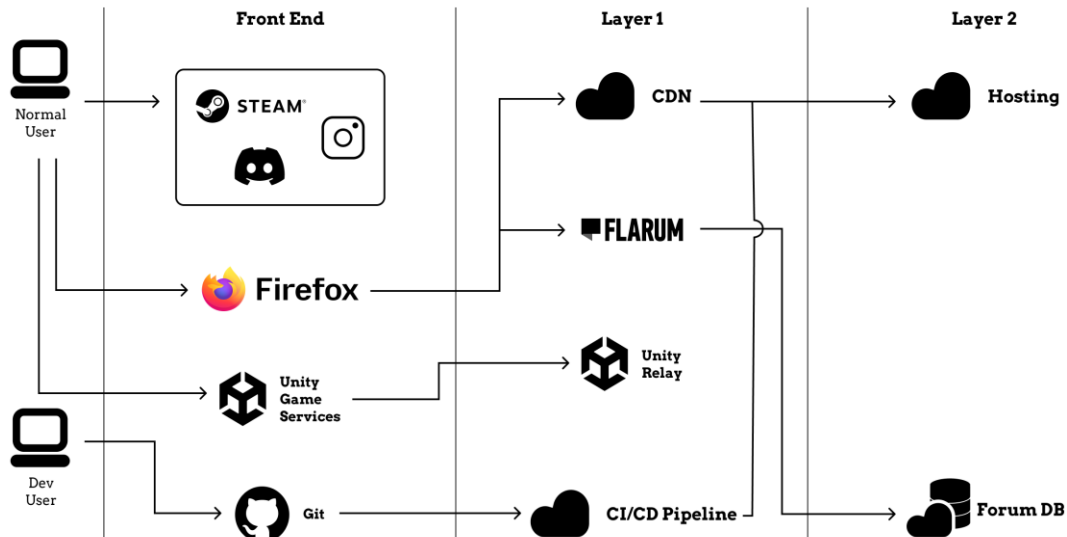
Punchline will include an achievement system, described in detail in Section 5.4.5, Achievements of the GDD.

To implement this feature, the game will rely on the native achievement systems provided by the target distribution platforms where available. These include:

- **Steam Achievements** for the PC version.
- **PlayStation Trophies** for PlayStation.
- **Xbox Achievements** for Xbox.

This approach allows players to track progress, unlock rewards, and showcase their accomplishments within their respective gaming profiles and reduces development overhead by avoiding the need for a fully custom achievement infrastructure while maintaining compatibility with platform standards.

9. Connection



9.1 Global Infrastructure

Punchline uses a host-client peer-to-peer architecture powered by Unity Netcode for GameObjects. There is no dedicated game server processing real-time gameplay data. Instead, one player's machine acts as the session host, authoritative over game state, while all other players connect as clients and send their inputs to the host for processing and redistribution.

9.1.1 Development Team Connectivity

The development team works fully remotely, with each member connecting to the shared infrastructure independently over their own internet connection. All collaboration and delivery flows through cloud-hosted services that require no VPN or shared local network:

- **Source control:** Every team member interacts with the Git repository hosted on GitHub or GitLab via HTTPS or SSH. Pushes to feature and develop branches trigger the CI/CD pipeline automatically, regardless of where the developer is located.
- **CI/CD pipeline:** Build and test runs execute on the cloud infrastructure of the chosen CI/CD provider (GitHub Actions or GitLab CI). Developers monitor pipeline status and retrieve build artifacts through the provider's web interface or CLI, with no need for a shared build machine.
- **Unity Gaming Services:** UGS (Relay, Lobby) is accessed by both the game clients at runtime and by developers during local testing via the UGS SDK, authenticated through a Unity project ID and environment key stored as CI/CD environment variables. No team member needs to manage UGS infrastructure directly.
- **Web hosting:** The CDN provider (Cloudflare, Netlify, or Vercel) is connected directly to the Git repository via OAuth. Deployments are triggered automatically on merge and require no manual intervention from the team.
- **Community forum:** The forum host (cloud VM) is accessed by authorized team members via SSH for maintenance and moderation tasks. No physical access to a shared machine is required.

All credentials and access keys are managed through the Git platform's secrets management system (GitHub Secrets or GitLab CI Variables), ensuring that no sensitive data is stored in the repository itself and that access can be revoked or rotated centrally.

9.1.2 Player Session Infrastructure

Session establishment follows this flow:

1. The hosting player creates a lobby via Unity Lobby, which generates a shareable join code.
2. Other players enter the join code to locate the session.
3. Unity Relay allocates a relay endpoint and provides both the host and all joining clients with a relay join code.
4. All peers connect to the relay endpoint. If a direct peer-to-peer connection can be negotiated (via UDP hole-punching), Unity NGO will prefer it; otherwise, traffic is routed through the relay with minimal overhead.
5. Once all players are connected, the host starts the game session, and NGO begins synchronizing object states across all clients.

The relay infrastructure is fully managed by Unity and distributed across multiple global regions. The UGS SDK automatically selects the lowest-latency relay region for the host, which indirectly benefits all clients connecting to that relay.

For the full release, platform-native session services (Steam Lobbies, PlayStation Network session management, Xbox Live sessions) will be integrated alongside or in place of Unity Lobby, depending on platform certification requirements.

9.2 Network Requirements

Punchline's gameplay is built around fast, coordinated combat. The quality of the peer-to-peer connection directly affects how fair and responsive the game feels. The following requirements define the network conditions needed to support an acceptable player experience:

Parameter	Target	Critical Threshold
Round-trip latency (RTT)	<80 ms	< 150 ms
Jitter	< 20 ms	< 40ms
Packet loss	< 1%	<3%
Upstream bandwidth	~50-150 Kbps	-

NAT Traversal: Unity Relay handles NAT traversal transparently for most configurations. In cases where direct UDP hole-punching succeeds, the relay is bypassed and the connection uses the lower-latency direct path. Players behind symmetric NAT will always route through the relay.

Session Stability: The NGO session manager handles brief network interruptions with a configurable client timeout window. A grace period of at least 5-10 seconds should be configured before a client is considered disconnected, preventing accidental drops due to transient network issues.

Host Migration: In the current host-client topology, if the host disconnects the session ends for all clients. For the vertical slice this is acceptable behavior. A host migration system, that transfers session authority to another connected peer, is planned as a post-vertical-slice feature to improve resilience in the full release.

Punchline: A literal Adventure

Università degli Studi di Milano - OGD - aa 2025-26



Connection Quality Feedback: Players should receive in-game feedback on connection quality (e.g. a latency indicator or packet loss warning) so they can make informed decisions about continuing or abandoning a session with poor network conditions. This data is available directly from the NGO transport layer.

10. Delivery

This section describes how Punchline will be delivered to players across the main stages of development.

It covers the planned release timeline, the delivery channels used for testing and public distribution, and the main activities that will follow each release phase.

10.1 Estimated Delivery Time

Punchline will not be released all at once, but through a series of progressive stages.

The first step will be an internal prototype to test the main combat mechanics and base of the game that will be developed in 2-3 months.

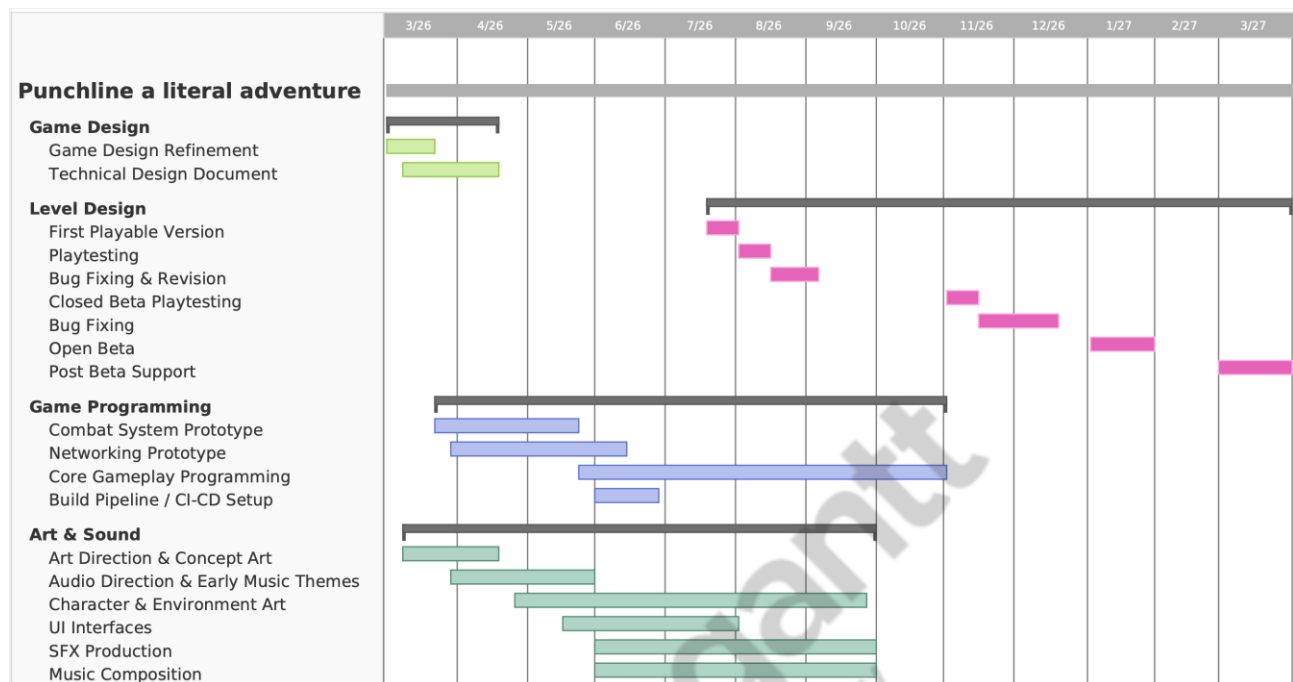
Then we will prepare a closed beta build for a limited group of testers, mainly to check gameplay quality, session stability, and the delivery workflow. The closed beta will be built in 5-6 months.

After that, we plan to move to an open beta (ready 1 month after the release of the closed beta) with more players, so we can collect wider feedback and fix the most important issues.

The full release will happen only after the beta phase gives good results.

Future updates or extra content may be added later depending on the state of the project.

Name	Start	Finish
Game design	2/3/2026	20/3/2026
Technical design document	20/3/2026	17/4/2026
Art Direction & Concept Art	9/03/2026	17/04/2026
Combat System Prototype	23/3/2026	22/05/2026
Network Prototype	30/3/2026	12/6/2026
Audio Direction & Early Music themes	30/03/2026	29/05/2026
Core Gameplay Programming	25/5/2026	30/10/2026
Character & Environment Art	27/04/2026	25/09/2026
Interfaces	18/05/2026	31/07/2026
SFX Production	1/6/2026	31/08/2026
Music Composition	1/6/2026	30/09/2026
Build Pipeline	1/6/2026	30/9/2026
First Playable Version	20/6/2026	31/7/2026
PlayTesting	03/8/2026	14/8/2026
Bug Fixing & Revision	17/8/2026	4/9/2026
Closed Beta PlayTesting	2/11/2026	13/11/2026
Bug Fixing	16/11/2026	18/12/2026
Open Beta	4/1/2027	29/1/2027
Post Beta Support	1/3/2027	31/3/2027



10.2 Delivery Platform

At this stage, Punchline will be delivered as a standalone PC build for internal testing and beta sessions. We chose this solution because it is more practical for the current development phase and lets us test the core gameplay and online features before moving to official store platforms.

For the open beta and the first public release, Steam will be our main platform on PC. It is the best choice for us because it already provides build distribution, updates, and visibility, which makes the delivery process easier to manage.

Other platforms like Nintendo Switch, PlayStation, Xbox, Epic Games Store, or GOG may be considered in a later stage of the project. For now, however, they are outside the scope of the vertical slice and of the current technical delivery plan.

10.3 Post-lunch plans

After the beta, our first goal will be to fix the main problems reported by players and make the game more stable. We want to improve the multiplayer experience, reduce technical issues, and make combat feel smoother and more balanced.

Once the PC version is stable enough, we may start adding small extra content and general improvements. These updates could include new encounters, visual polish, or other additions that can make the game more complete without changing its core structure too much. For now, the priority is not to expand the project too quickly, but to make sure that the current version works well and offers a solid experience to players. After that, we can evaluate bigger updates or the possibility of releasing the game on other platforms.

11. Cost Estimation

This chapter summarizes the main costs associated with the technical delivery of Punchline. Since the current project phase is focused on a PC vertical slice and the development workflow is fully remote, this estimate mainly considers operational services, external support, and the infrastructure required for beta and full release.

11.1.1 Staff intern

Role	Monthly cost (€)	Quantity	Months	Total (€)
Gameplay Programmer	1,500 €	3	8	€ 36,000
Network Programmer	1,700 €	3	8	€ 40,800
Technical Designer	1,200 €	2	8	€ 19,200
Art Director / Audio Designer	1,300 €	2	8	€ 20,800
QA Tester	900 €	2	2	€ 3,600

11.1.2 External services/outsourcing

Services	Unite cost	Quantity	Duration	Total
Unity Relay	25 / month	1	6 months	€ 150
Unity Lobby	10 / month	1	6 months	€ 60
Community Forum VM	15 / month	1	6 months	€ 90
Forum database	10 / month	1	6 months	€ 60
CI/CD Pipeline	4 / month	1	8 months	€ 32
Website Hosting/CDN	0 / month	1	8 months	€ 0
Legal support	1,500 / project	1	One-time	€ 1,500
Social Media Promotion	300 / month	1	4 Months	€ 1,200

11.1.3 Workspace

Item	Monthly cost	Quantity	Months	Total
Home Internet Contribution	25	4	8	€ 800
Shared Cloud Storage / Backup	10	1	8	€ 80
Miscellaneous Workspace Expenses	20	4	8	€ 640

11.1.4 One time setup costs

Item	Unit cost	Quantity	Total
Steam Direct Fee	100	1	€ 100
Visual Assets for Store Page	200	1	€ 200
Test Devices / Controllers	60	2	€ 120

11.2 Final cost evaluation

Category	Total
Internal Staff	€ 120,400
Esternal Services	€ 3,092
Workspace	€ 1,520
One-Time Setup	€ 420
Final total	€ 125,432